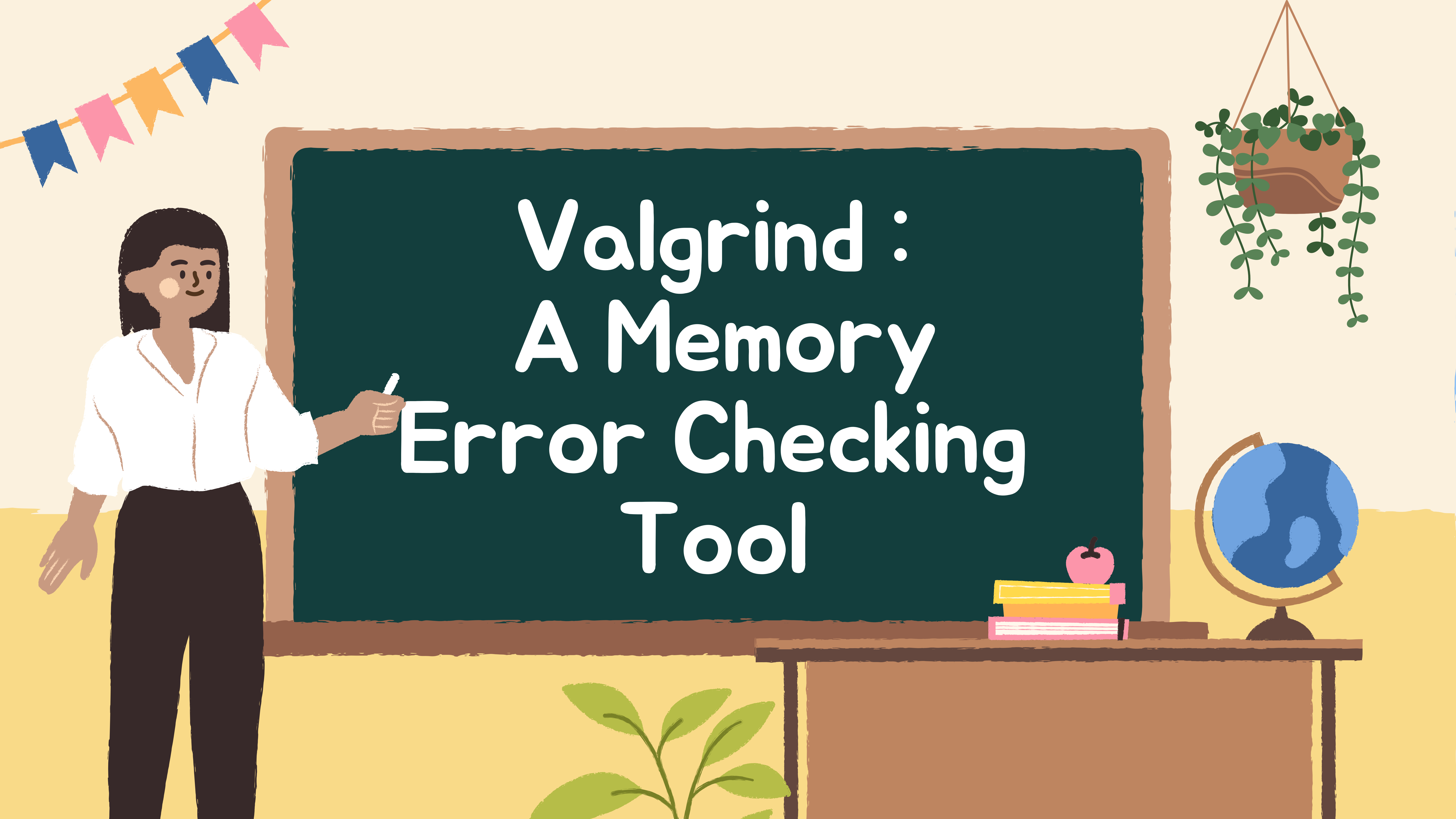

Valgrind :
A Memory
Error Checking
Tool

Presented by

Mahendra Kumar	23MCMC12
Prajwal Lute	23MCMC13
K. Saicharan	23MCMC14
Soumya Malkapure	23MCMC15

Outline

- What is Valgrind?
- What is a Memory Leak?
- Overview Of Memcheck
- Error Messages from Memcheck
- Memory Leak Detection with Memcheck
- Memcheck Command-Line Options
- Conclusion

What is valgrind

Valgrind is a programming tool suite designed for debugging and profiling applications. It primarily focuses on memory management debugging, memory leak detection, and profiling. Valgrind provides a framework for building dynamic analysis tools that can be used to analyze various aspects of a program's behavior.

What is Memory Leak

A memory leak refers to a situation where a computer program allocates memory (space) for temporary use, but fails to release it when it's no longer needed. It's like renting a storage unit but forgetting to return the key when you're done using it. As a result, that space remains unavailable for other tasks, leading to inefficient memory usage. Over time, if memory leaks accumulate, they can deplete the available memory resources, causing the program or system to slow down, become unresponsive, or even crash.

Overview Of Memcheck

Memcheck is a memory error detector. It can detect the following problems that are common in C and C++ programs.

- Accessing memory you shouldn't, e.g. overrunning and underrunning heap blocks, overrunning the top of the stack, and accessing memory after it has been freed.
- Using undefined values, i.e. values that have not been initialised, or that have been derived from other undefined values.
- Incorrect freeing of heap memory, such as double-freeing heap blocks, or mismatched use of malloc/new/new[] versus free/delete/delete[]

.

Overview Of Memcheck

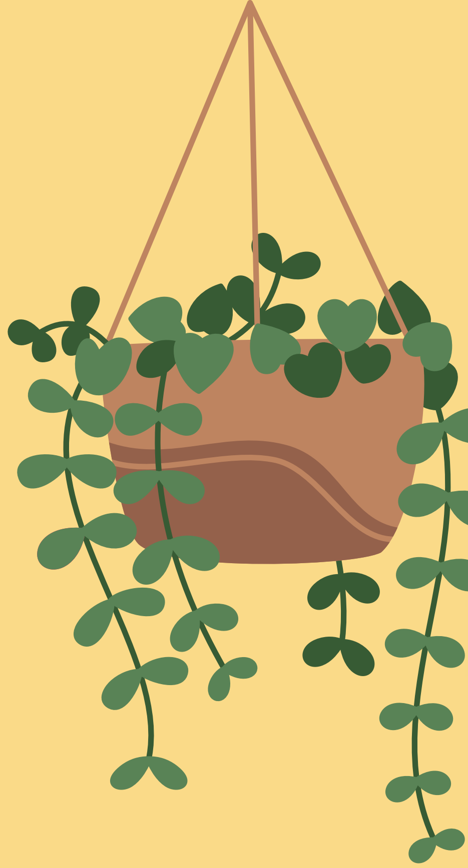
- Mismatches will also be reported for sized and aligned allocation and deallocation functions if the deallocation value does not match the allocation value.
- Overlapping src and dst pointers in memcpy and related functions.
- Passing a fishy (presumably negative) value to the size parameter of a memory allocation function.
- Using a size value of 0 with realloc.
- Using an alignment value that is not a power of two.
- Memory leak

Common Error Message from Memcheck

- Uninitialized Value: Detected when accessing variables without initialization, leading to unpredictable behavior or crashes.
- Memory Leak: Flags memory allocations not freed before program exit, causing gradual resource consumption and potential crashes.
- Invalid Memory Access: Highlights attempts to read/write memory without proper permission, such as out-of-bounds array access or dereferencing null pointers.
- Mismatched Allocation/Deallocation: Warns when memory is allocated with one function and deallocated with another, potentially leading to leaks or crashes.

Common Error Message from Memcheck

- Illegal Instruction: Reports execution of instructions violating CPU constraints or accessing invalid memory addresses.
- Conditional Jump on Uninitialized Value: Flags branches depending on uninitialized variables, leading to unexpected behavior.
- memcpy() Overlapping Buffers: Detects potential memory corruption when source and destination buffers overlap in memcpy() operations.
- Possible Data Race: Highlights concurrent memory access without proper synchronization, potentially causing non-deterministic behavior.

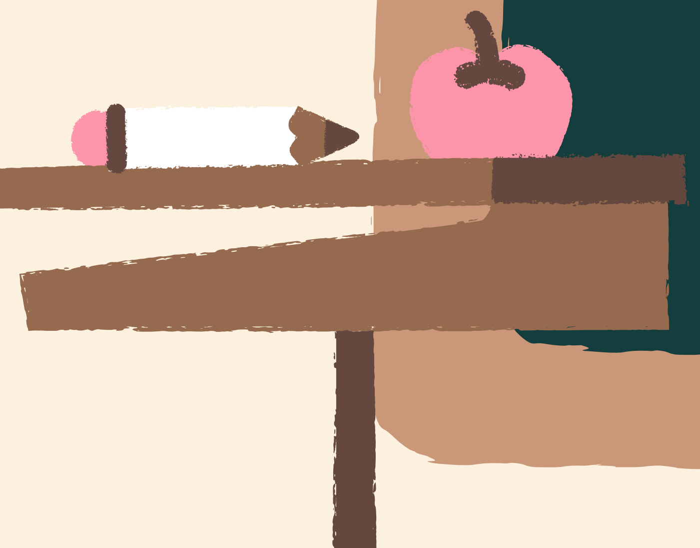


Memory Leak Detection with Memcheck

Memory leaks occur when a program fails to release memory it no longer needs, leading to gradual resource depletion and potential performance issues. Memcheck, a tool within Valgrind, helps detect and diagnose such memory leaks. Here's how it works:

1. Run Program with Memcheck: Execute your program through Valgrind with Memcheck enabled.

For example: `valgrind --tool=memcheck ./your_program`

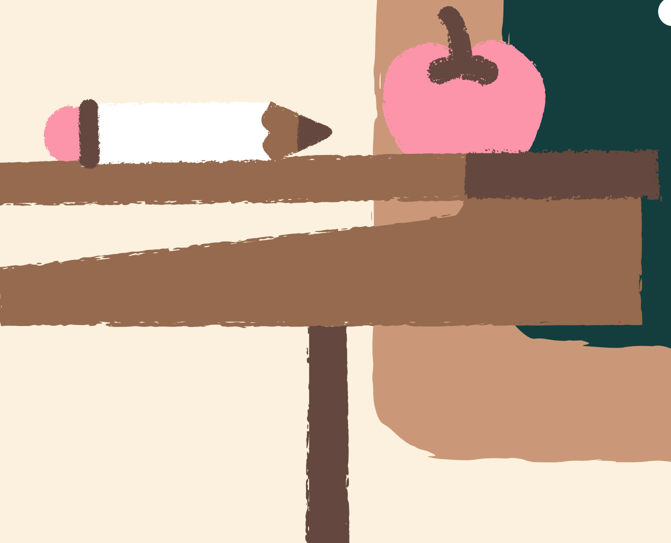


Memory Leak Detection with Memcheck

- Monitor Memory Operations: Memcheck tracks all memory operations during program execution, including allocations and deallocations.
- Identify Leaked Memory: After program execution, Memcheck analyzes memory usage and identifies any unreleased memory blocks.
- Report Leaks: Memcheck provides detailed reports highlighting memory leaks, including the location in the code where memory was allocated but not freed.

Memory Leak Detection with Memcheck

- Review Output: Examine Memcheck's output to understand where memory leaks occur and which functions are responsible. Look for patterns or common sources of leaks.
- Fix Leaks: Once identified, address memory leaks by properly deallocating memory using functions like free() in C or delete in C++. Ensure that memory is released under all execution paths.
- Re-run with Memcheck: After fixing potential leaks, re-run your program under Memcheck to verify that memory leaks have been resolved.



Memcheck Command-Line Options

- --leak-check=<yes|no>: Enables or disables memory leak checking. Use --leak-check=yes to enable leak checking (default), and --leak-check=no to disable .

- --show-leak-kinds=<[definite|possible|reachable|all]>: Specifies which types of leaks to report. Options include:
    - definite: Reports blocks definitely lost (default).
    - possible: Reports blocks possibly lost.
    - reachable: Reports blocks still reachable.
    - all: Reports all types of leaks.

Memcheck Command-Line Options

1. --track-origins=<yes|no|full|none>: Controls whether Memcheck tracks the origin of uninitialized values. Use --track-origins=yes to track origins (default), --track-origins=no to disable tracking, --track-origins=full for full tracking including stack traces, and --track-origins=none to track only uninitialized values without origins.
2. --show-reachable=<yes|no>: Enables or disables reporting of still-reachable blocks. Use --show-reachable=yes to include still-reachable blocks in leak reports (default), and --show-reachable=no to exclude them.

Conclusion

Valgrind's Memcheck tool, highlighting its significance in detecting various memory-related problems. It outlines how Memcheck can be used to identify issues like memory leaks, uninitialized memory accesses, and other memory errors. The tool's capabilities and usage are discussed, emphasizing its importance for improving program reliability and stability.

Thank You